DS 6021: Intro to Predictive Modeling
Lab 7: Missing Data Pt. 2, Multinomial Logistic Regression, KNN

November 6, 2026
Due: November 10, 2026 11:59pm

Materials Required

- framingham.csv – Framingham heart study dataset, predicting coronary heart disease
- glass.csv – A new dataset from the UCI repository, for doing classification on different glass fragments.
- Notebooks in our class github repository:
    - missing_data_and_imputation.ipynb
    - multinomial_KNN.ipynb

For Part 1 of this Assignment, we will do additional exploration around missing data, continuing one of the questions from Lab Activity 6. This will use the Framingham heart study data.

For Parts 2 and 3, we will use the UCI glass dataset that classifies different glass fragments to explore multinomial logistic regression and KNN in a bit more detail.

In class, we used multinomial logistic regression and KNN to predict the cultivar of 178 wines from 13 chemical measurements. Both models did very well, and most of the "competition" between them came down to misclassification of just a few wines. The wine classes are close to perfectly separable, which is why we saw these results.

With the glass dataset, the problem is a bit more challenging. There are six glass types instead of three, and the classes are very unbalanced: the largest has 76 fragments and the smallest has 9. Here, there is more of a difference between KNN and multinomial logistic regression, and we'll also explore in a bit more detail why we must be careful with accuracy as a metric.

Part 1: Missing Data and Imputation (7 pts)

In the last homework assignment, we spent a bit of time looking at the missing_data_and_imputation.ipynb notebook that we ran out of time to complete in class. In this exercise, I'll ask you to continue and try another approach for imputation. All of the code that you need should be in this notebook.

To turn in this portion of the assignment, you will have to write some code beyond what is already in this notebook, which we did not have to do last week. I recommend creating a copy and doing Part 1 of this assignment in that notebook, and then Parts 2 and 3 of this assignment in a separate notebook.

1. **[7 pts]** In Approach #4 in the notebook, we use stochastic regression imputation. Rather than imputing the mean value for each variable, we predict sysBP with age, cigsPerDay and BMI as features. In this part of the notebook, I'd like you to repeat this analysis using totChol as a predictor of sysBP. Yes, you read that right, I am telling you to use the target variable as a predictor for this portion of the exercise! *Note: For the cell numbering I am listing below, I commented out the code in cell 4 that errors out, so I could run the notebook straight through from start to finish.*

    a. [**2 pts**] First, look at the code for Approach #4. Change the noise seed in cell 15 to a different value, and rerun the remainder of the notebook. Does the value of coefficient estimate for the sysBP change? What is this telling us? (Note: We have kept WHICH values of sysBP are missing, because the deletion in cell 7 uses its own seed)

    b. [**3 pts**] Repeat Approach #4, but make the following changes:
        i. cell 14: Add a second formula to predict sysBP: `formula2 = "sysBP ~ age + cigsPerDay + BMI + totChol"`
        ii. cells 14+15: Add a second imputer model using formula2 and use it to impute sysBP in a copy of the data. The goal here is to compare this imputer with the previous one. We want to save all of our previous output!
        iii. Add this model, model5 to cell 16 and the plot in cell 17. You should now have a plot comparing 5 models: Full model, missing data, mean imputation, regression imputation, regression imputation with target.
    c. [**2 pts**] What do you observe about the imputation with the target variable? How is it different from the first regression imputation that did not include it?

Part 2: Multinomial Logistic Regression (16 pts)

Use the following setup for Parts 2 and 3 of this lab. Please use the same random seeds for consistency in grading, and use the same CV object for each model. You might also find the `cv_scores` and `cv_pred` functions from the in-class notebook useful.

```
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate, cross_val_predict
from sklearn.metrics import log_loss, accuracy_score, precision_score, recall_score, ConfusionMatrixDisplay

warnings.filterwarnings("ignore")   # This suppresses some of the warning output in the saga solver

col_names= ["RI", "Na", "Mg", "Al", "Si", "K", "Ca", "Ba", "Fe", "type"]
glass = pd.read_csv("../../data/glass.csv", header=None, names=col_names)

class_names = {1: "building_float", 2: "building_nonfloat", 3: "vehicle_float",
               5: "containers", 6: "tableware", 7: "headlamps"}
features = ["RI", "Na", "Mg", "Al", "Si", "K", "Ca", "Ba", "Fe"]

X = glass[features]
y = glass["type"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=113
)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=224)
```

1. **[5 pts] Data exploration and model baselines**

    a. Explore the data and make a table looking at the mean, standard deviation, min and max for each of the features.

    b. Make a table showing how many glass fragments of each type are in the training set and in the test set. Which classes are rare? Which is the most common?

    c. On the training set, what is the accuracy of a "model" that always predicts the most common class?

    d. Compute the base-rate log loss on the training set. You will want to predict the training class proportions for every glass type, as we did in class.

    e. In the in-class deck, Classification_and_GLMs, I note on slide 56 that predicting 1/K for every class gives a log loss of ln K. For this particular dataset, that would be ln 6. Why is the base-rate log loss from part (d) lower than ln 6?

2. **[4 pts] Build an unpenalized model (no regularization)**

    a. The eight oxide columns are weight percentages. (Oxides are every feature but RI) Add them up across each row and summarize the result. What do you find?

b. Fit a multinomial logistic regression with no penalty, `LogisticRegression(C=np.inf, max_iter=50000)`, in a pipeline with `StandardScaler()`. Use cross-validation to report the mean log loss and accuracy, then fit the model on the full training set and look at its coefficients.

c. Compute the VIF for each of the nine predictors (as we did in Lab 3). What does this tell us about the coefficients we calculated in Part b, above? Relate your answer to what you found in Part a.

d. Compare this model's CV log loss with the base-rate log loss from Q1d. How can a model with over 60% accuracy have a *worse* log loss than a model that uses no predictors at all?

3. **[5 pts] Build a regularized multinomial logistic regression model.** Using the same `C_values`, the same `l1_ratios`, and the same "simplest model within one standard error" rule as the in-class notebook, build multiple regularized models and choose the best one. Remember that to find the simplest model within one standard error of the minimum log loss we had to fit the candidate models on the full training set to find the number of non-zero coefficients. All of these steps can be found in our in-class notebook. Note: In class we printed out a table looking at log loss for each. You don't need to do that here. You should create the table, but you don't have to print it.

   a. Plot the mean CV log loss against C for each penalty type, with the base-rate log loss as a reference line. Which penalty and C give the lowest CV log loss? Which model does the one-standard-error rule select, and how many of its 54 coefficients are nonzero?

   b. Plot the mean CV accuracy of each model against C for each penalty type. (This is the same plot as in part a, just using accuracy instead of log loss.) Which C has the highest accuracy? Compare with the log loss and accuracy of the model in part a.

   c. For large values of C ($C \geq 10$), every log loss curve sits above the base-rate line. Explain why, including what large C means and your answers in Q2. Why is accuracy higher than the baseline for large C, but log loss gets worse?

   d. Fit the selected model from part a on the full training set and make a heatmap of its coefficients. Which features have a nonzero coefficient for only one class?

   e. By what factor does a one standard deviation increase in Ba multiply the odds of *headlamps* vs. *building_float*? Repeat for Mg, comparing *building_float* vs. *headlamps*.

4. **[2 pts] Where does the model make mistakes?** Use `cross_val_predict` to make a pooled out-of-fold confusion matrix for the selected model, and calculate precision and recall for each class in a table.

   a. Which classes does the model almost never predict correctly?

b. Most vehicle_float fragments are predicted as which class? Using the class descriptions on the UCI dataset page, why might these two classes be hard to tell apart?

Part 3: KNN + Model Comparisons (15 pts)

1. **[4 pts] Choosing k and the distance metric.** In this part of the assignment, we will look at two different distance metrics and a variety of different values for k. Recall that sklearn's p argument in the KNeighborsClassifier class sets the Minkowski exponent, where p = 1 is Manhattan distance, p = 2 is Euclidean. For k = 1, 3, 5, …, 31 and p ∈ {1, 2}, compute the mean CV log loss and accuracy. Plot both metrics against k, with one line for each value of p.

   a. Which (k, p) gives the lowest CV log loss?

   b. If you had chosen k by accuracy instead, which (k, p) would you have picked? Explain why that k has one of the best accuracies but by far the worst log loss, and why it would be a poor choice.

   For the rest of this question, use the (k, p) with the lowest CV log loss, as we did in class.

2. **[2 pts] Where does KNN's log loss come from?** Just as we can do for logistic regression, we can look at which KNN predictions contribute the most to log loss. To do this, use `cross_val_predict(..., method="predict_proba")` to get out-of-fold predicted probabilities for the KNN model chosen in Q1. For each row you will get 6 predicted probabilities: 1 for each class. Pick the probability that corresponds to the true class. This is the only one that is counted in the log loss.

   a. How many fragments get a probability of exactly 0 for their true class? Which classes are they from?

   b. Before taking the log, sklearn replaces a probability of 0 with a tiny number, `np.finfo(float).eps` (about 2.2×10^{-16}). For the fragments identified above in Part a, how much does each one add to the log loss? What percentage of the total KNN log loss comes from these fragments? What is the mean log loss of the remaining fragments?

3. **[6 pts] Comparing the two models**

   a. Compare the CV log loss and CV accuracy of your selected logistic model and your selected KNN. Make a second table that shows recall for each class (`recall_score(..., average=None)`) for both models. Which model has the lower CV log loss?

   b. Compare the confusion matrices for each model. Using the confusion matrices for each model, explain where KNN's extra accuracy comes from and why it isn't a good reason to prefer KNN.

c. Using the recall table, which glass classes does KNN handle better, and which does it not help? What does this suggest about the shape of the boundaries between some of the glass classes?

4. **[3 pts] Evaluate on the test set.** Refit both models on the full training set, then report the test log loss, the test accuracy, and a test confusion matrix for each.

   a. [1 pt] Do the test results rank the two models the same way cross-validation did?

   b. [2 pts] The test set has only 43 fragments, with 2 tableware fragments and 3 each of vehicle_float and containers. What can and can't you conclude about per-class performance from these results? How much would a single fragment with a predicted probability of 0 for its true class add to KNN's test log loss?

Deliverables

For each part, please include the following:

- All code used to complete the activities. This can be turned in as a single Jupyter notebook, multiple Jupyter notebooks, or one or more .py files. Up to you!
- For open ended questions, please include a few sentences or a paragraph answering the question. This text can be included in your Jupyter notebook as a markdown cell or a separate document.
- Per our policy on Generative AI detailed in the syllabus, if you use it to generate code, you need to flag how and where you used it, and how you ascertained the correctness of what you did. You are not allowed to use Generative AI for any of the written explanations or reflection questions. If you did not use Generative AI for any part of the assignment, please state that explicitly.